

Ejemplo: 8 Puzzle

7	2	4
5		6
8	3	1

Start
State

	1	2
3	4	5
6	7	8

Goal
State

$$S = (S_0, \dots, S_8)$$

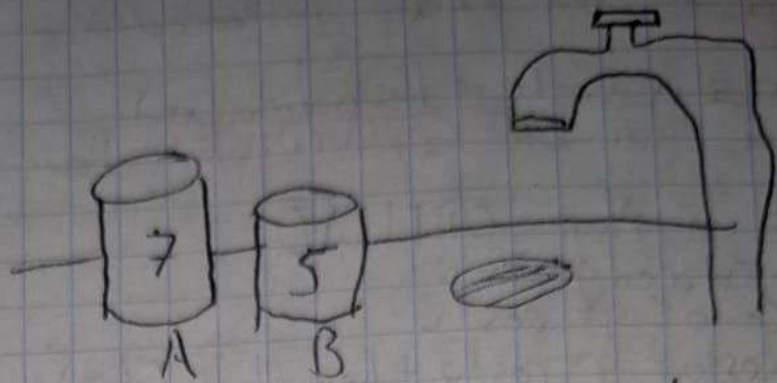
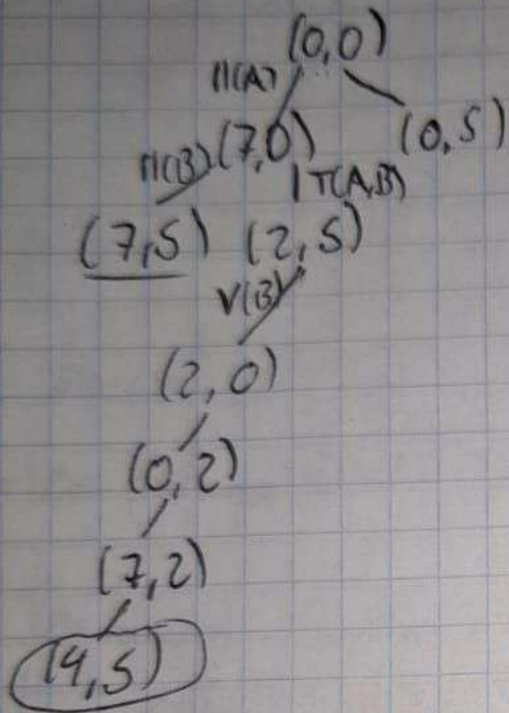
$$S_i \in \{0, \dots, 8\}$$

$$\text{acciones} = \{T, U, \rightarrow, \leftarrow\}$$

$$\text{card}|S| = \frac{9!}{2}$$

$h_1 = \#$ de piezas mal colocadas
 $h_2 = \text{dist. Manhattan de cada pieza a su posición}$

Profonde



4 lts en un bote

Heurística trivial, Dominancia

$$h(n) = 0 \quad \forall n$$



$h(n) + n.\text{costo}$ es el costo exacto de la solución óptima que n sea un Plan intermedio

Si

$$h_1(n) \leq h_2(n)$$

$\forall n$ entonces

h_2 domina a h_1

¿Cuáles serían dominantes respecto a h_1 y h_2 ?

a) $h(n) = \min(h_1(n), h_2(n))$

ADMISIBLE

b) $h(n) = h_1(n) * h_2(n)$

c) $h(n) = \max(h_1(n), h_2(n))$

ADMISIBLE

d) $h(n) = (h_1(n) + h_2(n)) / 2$

ADMISIBLE

h es Admisible si:

$$h(n) \leq g^*(n)$$

donde $g^*(n)$ es el costo del plan óptimo iniciando con n en n . estado

$$h(n) = 0 \text{ si } n. \text{ estado es terminal}$$

Yo en A^* ordena los nodos por $n.\text{costo} + h(n)$ si n . estado es terminal $n.\text{Costo} + h(n) = n.\text{Costo}$

Para que sea dominante ocupa ser siempre o mayor o igual, por lo tanto $\max(h_1(n), h_2(n))$ siempre va a ser mayor o igual que h_1, h_2 .

Planificación: ir de un estado inicial a un estado final, con el menor costo posible.



Juegos

Juego Por turnos 2 jugadores

class Juego2TED (Object):

```

def acciones_legales(self, estado, jugador):
    return conjunto acciones
def sucesor(self, estado, accion, jugador):
    return estado-nuevo
def terminal(self, estado):
    return bool
def ganancia(self, estado):
    return numero
  
```

Ej. Juego del gato

0	1	2
3	4	5
6	7	8

$S = (S_0, \dots, S_8)$

$S_i \in \{1, 0, -1\}$

$a \in \{0, \dots, 8\}$

Jugador 1

Jugador 2

En caso que no haya nada

```

class Gato(juego2D):
    def acciones legales (self, estado, jugador):
        return [i for (i,s) in enumerate(estado) if s == 0]
    def sucesor (self, estado, accion, jugador):
        s-n = estado[i]
        s-n[accion] = jugador
        return s-n
    def ganancia (self, estado):
        if 0 not in estado:
            return 0
        if ((s[0] == s[4] == s[8]) or
            (s[2] == s[4] == s[6])) and s[4] != 0:
            return s[4]
        for i in range(3):
            if ((s[i] == s[i+3] == s[i+6]) or
                (s[3*i] == s[3*i+1] == s[3*i+2])) and s[3*i] != 0:
                return s[3*i]

```

Minimax Implementation

$a^* = \max(e)$ For e in $\text{juego. acciones_legales}(s, j)$,
 $\text{Key} = \text{Valor}(\text{juego. Sucesor}(s, a, j), -1) * j$

def value(state):
if the state is a terminal state: return the state's utility
if the next agent is MAX: return max-value(state)
if the next agent is MIN: return min-value(state)

def max-value(state):
initialize $v = -\infty$
for each successor of state:
 $v = \max(v, \text{value}(\text{successor}))$
return v

def min-value(state):
initialize $v = +\infty$
for each successor of state:
 $v = \min(v, \text{value}(\text{successor}))$
return v

Alpha-Beta Implementation

α : MAX's best option on path to root
 β : MIN's best option on path to root

def max-value(state, α , β):
initialize $v = -\infty$
for each successor of state:
 $v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$
 if $v \geq \beta$ return v
 $\alpha = \max(\alpha, v)$
return v

def min-value(state, α , β):
initialize $v = +\infty$
for each successor of state:
 $v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$
 if $v \leq \alpha$ return v
 $\beta = \min(\beta, v)$
return v